# Python Data Worksheet: From Penguins to Predictions

You will load a real dataset, analyse it with pandas, plot it with matplotlib, and finish by building two small machine learning models yourself.

## Setup

You need Python 3 with three libraries: **pandas** (tables of data), **matplotlib** (graphs) and **numpy** (fast maths on arrays). Install them from a terminal:

```
pip install pandas matplotlib numpy
```

No installation possible? Use Google Colab in the browser. The libraries are already there.

**The dataset.** You will work with the Palmer Penguins dataset: 344 penguins from three species (Adelie, Chinstrap, Gentoo) on three islands in Antarctica. For each penguin it records bill length and depth, flipper length, body mass and sex. Download it as a CSV file, or let pandas read it straight from the web:

```
URL = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/penguins.csv"
```

| Column | Type | Meaning |
|---|---|---|
| `species` | text | Adelie, Chinstrap or Gentoo |
| `island` | text | Biscoe, Dream or Torgersen |
| `bill_length_mm` | number | Length of the beak, in mm |
| `bill_depth_mm` | number | Height of the beak, in mm |
| `flipper_length_mm` | number | Length of the wing, in mm |
| `body_mass_g` | number | Weight, in grams |
| `sex` | text | MALE or FEMALE (sometimes missing) |

Work in a Jupyter notebook or a `.py` file, whichever you prefer. Each exercise builds on the previous one, so keep your code.

## Exercise 1 — Meet the penguins: loading and exploring data

> ***Theory box: pandas DataFrames***

- A **DataFrame** is a table: rows are observations (one penguin), columns are variables (`body_mass_g`). Each column on its own is a **Series**, a one-dimensional array with an index.
- `pd.read_csv(path_or_url)` reads a CSV file into a DataFrame. It guesses each column's type (`dtype`): numbers become `float64`/`int64`, text becomes `object`.
- Selecting: `df["species"]` gives a Series; `df[["species", "sex"]]` (a list of names) gives a smaller DataFrame.

- Inspecting: `df.shape` is `(rows, columns)`; `df.head(n)` shows the first n rows; `df.info()` lists every column with its dtype and how many values are not missing; `df.describe()` gives count, mean, standard deviation, min, quartiles and max for each numeric column.
- **Missing values** appear as `NaN` ("not a number"). `df.isna()` gives a table of `True`/`False`; `.sum()` counts the `True` values per column. `df.dropna()` removes every row with at least one `NaN`; `df.dropna(subset=["a", "b"])` only looks at the listed columns.
- **Vectorised operations:** maths on a column works on every row at once, with no loop: `df["mass_kg"] = df["body_mass_g"] / 1000` creates a new column.

```
import pandas as pd

URL = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/penguins.csv"
df = pd.read_csv(URL)

print(df.shape)
print(df.head())
df.info()
```

**Your tasks:**

1. Load the dataset and print its shape. How many penguins and how many columns are there?
2. Find out which columns have missing values, and how many in each.
3. Make a cleaned table `df_clean`. Decide for yourself: drop every row with any `NaN`, or only rows missing the four measurements? Print how many rows each choice removes.
4. Add two new columns without using a loop: `body_mass_kg`, and `bill_ratio` = bill length ÷ bill depth.
5. Use `describe()`. Which measurement varies most compared to its own size? Compute the coefficient of variation (standard deviation ÷ mean) for each numeric column to answer it.

**Checkpoint:** 344 rows and 7 columns. Two penguins have no measurements at all; 11 are missing `sex`.

**Think about it:** If you study body mass by sex, you must drop the rows without `sex`. If you study bill size by species, should you drop them too? What do you lose if you always use `dropna()` without thinking?

## Exercise 2 — Asking questions: filtering and grouping

> ***Theory box: boolean masks and split–apply–combine***

- A comparison on a column gives a **boolean mask**, a Series of `True`/`False`: `df["species"] == "Gentoo"`. Putting it inside `df[...]` keeps only the `True` rows.
- Combine masks with `&` (and), `|` (or), `~` (not). Each condition needs its own brackets: `df[(df["sex"] == "FEMALE") & (df["body_mass_g"] > 4000)]`. Python's `and`/`or` do not work here.
- `df.loc[mask, ["col1", "col2"]]` selects rows and columns at the same time. `df.sort_values("col", ascending=False)` sorts; `df["col"].idxmax()` gives the index of the largest value.
- `df["species"].value_counts()` counts how often each value appears. `pd.crosstab(df["species"], df["island"])` counts every combination of two columns.
- **groupby** follows the pattern *split → apply → combine*: split the rows into groups, compute something per group, join the results into a new table.

`df.groupby("species")["body_mass_g"].mean()` gives one mean per species. Group by several columns with a list, and compute several statistics with `.agg(["mean", "std", "count"])`.

- Trick: the mean of a boolean column is the share of `True` values. `(df["body_mass_g"] > 4000).mean()` is the fraction of penguins over 4 kg.

```
# Average flipper length per species, longest first
(df.groupby("species")["flipper_length_mm"]
   .mean()
   .sort_values(ascending=False))
```

**Your tasks:**

1. How many penguins of each species are there? Which species live on which islands?
2. Find the heaviest penguin. Print its species, island, sex and mass.
3. Make one table with the mean, standard deviation and count of body mass for every combination of species and sex. Which group is heaviest on average?
4. How many female Adelie penguins from Torgersen have flippers longer than 190 mm?
5. Challenge: for each species, what percentage of penguins is heavier than the average penguin in the whole dataset? Do it in at most three lines, without a loop.

**Checkpoint:** 152 Adelie, 68 Chinstrap, 124 Gentoo. Gentoos live only on Biscoe and Chinstraps only on Dream; Adelies are on all three islands. The heaviest penguin weighs 6,300 g.

**Think about it:** Penguins on Biscoe are heavier on average than penguins on Dream. Does that mean Biscoe makes penguins heavy? Use your crosstab to explain what is really going on.

## Exercise 3 — Seeing the data: graphs with matplotlib

> ***Theory box: figures, axes and choosing a chart***

- A matplotlib plot has two layers: the **Figure** (the whole image) and one or more **Axes** (the individual plots inside it). `fig, ax = plt.subplots()` makes one of each; `fig, axes = plt.subplots(1, 3, figsize=(12, 4))` makes a row of three.
- You draw on an Axes: `ax.hist(values, bins=20)`, `ax.scatter(x, y)`, `ax.bar(labels, heights)`, `ax.plot(x, y)`. Then label it: `ax.set_xlabel(...)`, `ax.set_ylabel(...)`, `ax.set_title(...)`, `ax.legend()`.
- Pick the chart from the question: the **distribution** of one number → histogram; the **relationship** between two numbers → scatter plot; **comparing groups** → bar chart; change **over time** → line.
- To compare groups in one plot, loop over the groups and draw each with its own `label=`; `alpha=0.5` makes overlapping marks see-through.
- `fig.savefig("penguins.png", dpi=150, bbox_inches="tight")` saves the image; `plt.show()` displays it.
- Every good graph has axis labels **with units**, a title that says what it shows, and a legend when colours mean something.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
for species, group in df_clean.groupby("species"):
```

```
    ax.hist(group["body_mass_g"], bins=20, alpha=0.5, label=species)
ax.set_xlabel("Body mass (g)")
ax.set_ylabel("Number of penguins")
ax.set_title("Body mass by species")
ax.legend()
plt.show()
```

**Your tasks:**

1. Make a histogram of flipper length for all penguins together. How many peaks do you see? What could explain them?
2. Redraw it with one colour per species. Was your explanation right?
3. Make a scatter plot of bill length (x) against bill depth (y), coloured by species, with labelled axes and a legend.
4. Make a bar chart of the mean body mass per species, with male and female side by side. (Tip: the table from Exercise 2, task 3, with `.unstack()` and `.plot(kind="bar")`.)
5. Combine three of your plots into one figure with `plt.subplots(1, 3)` and save it as a PNG file.
6. Creative: design one graph that would convince a friend that Gentoos are clearly different from the other two species. Choose the columns and chart type yourself.

**Think about it:** Fit a straight line through all the points in task 3 at once, and it slopes downwards: longer bills seem to come with shallower bills. Now look at each species on its own. Which way do the points go? This is called **Simpson's paradox**. Why is it dangerous to analyse groups mixed together?

## Exercise 4 — Your first model: predicting weight with a straight line

> ***Theory box: linear regression and mean squared error***

- A **model** turns an input x into a prediction ŷ ("y-hat"). The simplest one is a straight line: ŷ = a · x + b, where a is the slope and b the intercept. Here x is flipper length and y is body mass.
- For each penguin the **residual** y − ŷ is how far the prediction is from the truth. Some are positive, some negative, so we square them before averaging. The result is the **mean squared error** (MSE), shown below.
- **Training** the model means choosing the a and b that make the MSE as small as possible ("least squares"). For a straight line there is an exact formula, shown below; x̄ and ȳ are the means of x and y.
- MSE is in squared units (g²), which is hard to read. Its square root, the **RMSE**, is back in grams: roughly, how wrong a typical prediction is.
- **numpy** does maths on whole arrays at once: `x = df_clean["flipper_length_mm"].to_numpy()`, then `np.mean((y - y_pred) ** 2)` is the MSE in one line. `np.polyfit(x, y, 1)` returns `[a, b]` for the best line, which you can use to check your own result.

Mean squared error over n penguins:

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

The best slope and intercept:

$$a = \frac{\sum_i (x_i - \bar{x})\,(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad b = \bar{y} - a\,\bar{x}$$

**Your tasks:**

1. Make a scatter plot of flipper length (x) against body mass (y). Does a straight line look like a reasonable model?
2. Write a function `mse(y, y_pred)` using numpy, with no loop.
3. Guess three lines by hand (three pairs of a and b). Draw them on top of the scatter plot and compute the MSE of each. Which guess is best?
4. Compute the best a and b with the formula above. Check them with `np.polyfit`, then draw the line in a different colour.
5. Predict the mass of a penguin with 200 mm flippers. Compute the RMSE and explain in one sentence what it means.
6. Challenge: train the line with **gradient descent** instead of the formula. Start with a = b = 0 and repeat many times: compute the predictions, then move a and b a small step against the gradient of the MSE ($\partial MSE/\partial a = -2/n \cdot \Sigma\, x_i(y_i - \hat{y}_i)$ and $\partial MSE/\partial b = -2/n \cdot \Sigma\,(y_i - \hat{y}_i)$). Hint: it only works well if you first standardise x, i.e. `x_s = (x - x.mean()) / x.std()`. Plot the MSE after every step. This is, in miniature, how neural networks learn.

**Checkpoint:** a ≈ 50 g per mm and b ≈ −5,780 g. The RMSE is about 390 g.

**Think about it:** Use your line to predict the mass of a penguin with 100 mm flippers. What goes wrong, and what does that tell you about using a model outside the data it was trained on?

## Exercise 5 — Which penguin is this? A classifier from scratch

> ***Theory box: classification with k-nearest neighbours***

- In Exercise 4 you predicted a number (**regression**). Now you predict a category, the species (**classification**). The inputs are called **features** (bill length, bill depth) and the answer is the **label**.
- **k-nearest neighbours (KNN)** needs no training at all. To classify a new penguin: (1) compute its distance to every penguin you already know; (2) take the k closest ones; (3) let them vote: the most common species wins. The distance is the ordinary Euclidean distance, shown below.
- **Train/test split:** a model must be judged on data it has never seen. Shuffle the rows (`df.sample(frac=1, random_state=42)`), keep 80 % as the training set (the "known" penguins) and hide 20 % as the test set.
- **Accuracy** = correct predictions ÷ all predictions. A **confusion matrix**, `pd.crosstab(true_labels, predicted_labels)`, shows which species get mistaken for which.
- **Feature scaling:** distance adds up differences, so a feature measured in thousands of grams drowns out one measured in tens of millimetres. Standardise each feature with z = (x − mean) ÷ standard deviation, using the training set's mean and standard deviation for both sets.

- Useful numpy: `np.sqrt(((X_train - point) ** 2).sum(axis=1))` gives the distance from one point to every training row at once; `np.argsort(d)[:k]` gives the positions of the k smallest distances.

Distance between penguins p and q with m features:

$$d(p,q) = \sqrt{\sum_{j=1}^{m} (p_j - q_j)^2}$$

**Your tasks:**

1. Use `bill_length_mm` and `bill_depth_mm` as features and `species` as label. Shuffle `df_clean` and split it 80/20 into `X_train`, `y_train`, `X_test`, `y_test` (numpy arrays).
2. Write a function `predict(point, X_train, y_train, k)` that returns the predicted species for one penguin.
3. Predict every penguin in the test set with k = 5. Compute the accuracy and the confusion matrix. Which species are mixed up most often?
4. Try k = 1, 3, 5, 11, 25 and 75. Plot accuracy against k. What happens when k is very small, and when it is very large?
5. Add `body_mass_g` as a third feature, first without scaling, then with standardisation. Compare the accuracies and explain the difference.
6. Creative challenge (pick one):
    - **Decision map:** use `np.meshgrid` to make a grid of bill lengths and depths, predict the species for every grid point, and colour the plane by prediction. Draw the real penguins on top. Where are the borders between species?
    - **Check against the pros:** train scikit-learn's `KNeighborsClassifier` (`pip install scikit-learn`) on the same split. Do you get the same accuracy as your own code?

**Checkpoint:** with the two bill features and k = 5, accuracy should be well above 90 %.

**Think about it:** Is 95 % accuracy good? Compute the accuracy of a "model" that always guesses the most common species. Any real model must beat this **baseline**. When would 95 % still not be good enough, for example in medicine?